Tracing Java Applications with AWS X-Ray

Description

In this lab, you will be instrumenting a Spring Application that you will create in Cloud9 IDE which has APIs to insert data in a DynamoDB table and fetch the data from the table. AWS X-Ray helps you analyze your applications, it will help you understand how your application is performing. You will be able to troubleshoot the errors very easily. There are multiple ways for how you can instrument your application. In this lab, you will be using manual instrumentation.

There are a few definitions you should understand before starting the lab:

- **Traces** are used to track the path followed by a request in your application.
- **Segments** provide basic details about the resource name, details about the request. Segments can be broken into **subsegments** to provide more granular details about the request.

Learning Objectives

Upon completion of this lab you will be able to:

- Insert data into DynamoDB tables
- Fetch data from DynamoDB tables
- Instrument applications with AWS X-Ray

Intended Audience

This lab is intended for:

- QA engineers
- Software Developers
- Anyone interested in debugging

Prerequisites

You should possess:

- A basic understanding of Java
- A basic understanding of the Spring Boot framework is helpful but not required
- A basic understanding of using AWS SDK is helpful but not required

The following labs can be used to fulfil the prerequisites:

- Coding with Java Language Basics
- Coding with Java Wrappers, Enums, and Dates

Updates

August 1st, 2025 - Updated screenshots & instructions, updated Maven installation step

June 6th, 2023 - Updated instructions to improve clarity

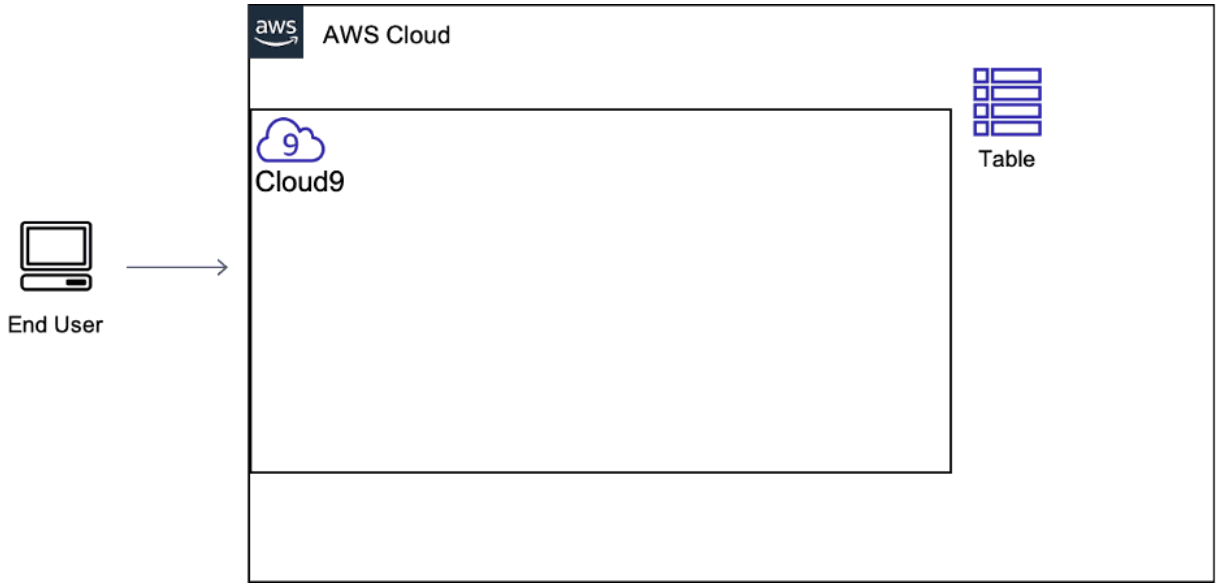
April 25th, 2023 - Updated screenshots & instructions to reflect the latest Cloud9 UI

April 13th, 2023 - Updated screenshots & instructions to reflect the latest UI

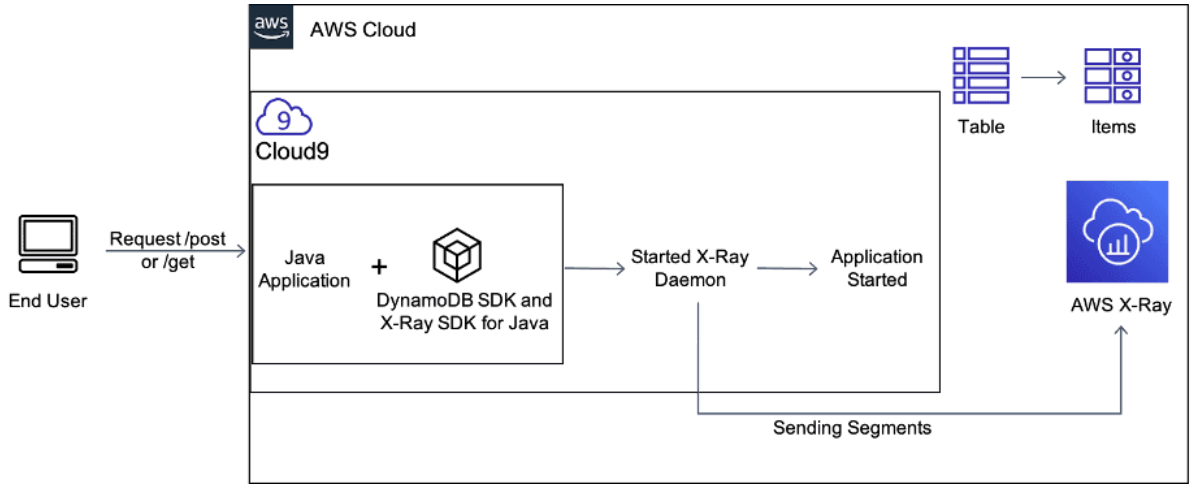
March 12th, 2023 - Resolved an issue that caused the lab to fail to set up on rare occasions

May 26th, 2022 - Improved the instructions and added a screenshot

Environment before



Environment after



Lab steps

0 of 7 steps completed.Use arrow keys to navigate between steps. Press Enter to go to a step if available.

- Logging In to the Amazon Web Services Console
- Opening the AWS Cloud9 IDE
- Installing the Required Tools
- Configuring the AWS DynamoDB SDK and AWS X-Ray SDK
- Initiating the DynamoDB Client and Configuring the Segments
- Starting X-Ray Daemon and Testing the Application
- Finding and Understanding the Traces in the AWS Console

Opening the AWS Cloud9 IDE

Introduction

This lab requires an integrated Development Environment (IDE) to instrument the Java application with AWS X-Ray. In this Lab, you will use AWS Cloud9 as the IDE.

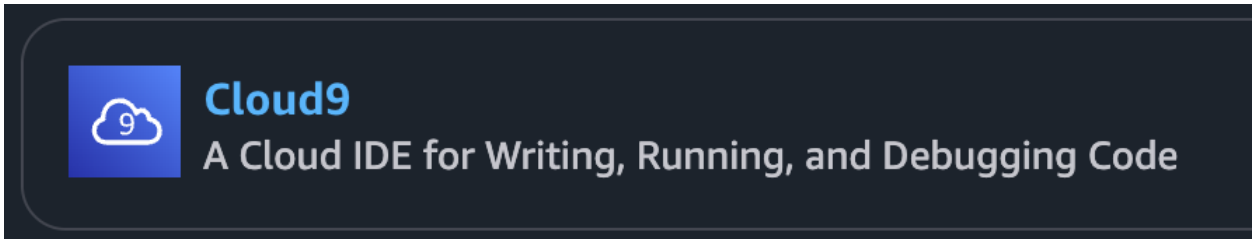
AWS Cloud9 is an Integrated Development Environment (IDE) that runs in a browser. You can write, run, and debug your code in Cloud9. The Cloud9 environment can be shared with multiple collaborators to work on the same project at the same time. You can view the changes collaborators are making in real-time, making it a useful tool for pair programming.

Cloud9 environments are backed by either an EC2 instance or by any ssh-connected machine with Python installed, including machines outside of AWS. The IDE includes an integrated terminal connected to the machine that backs the Cloud9 environment. For EC2 Cloud9 environments, Cloud9 will automatically put the EC2 instance to sleep after a period of inactivity

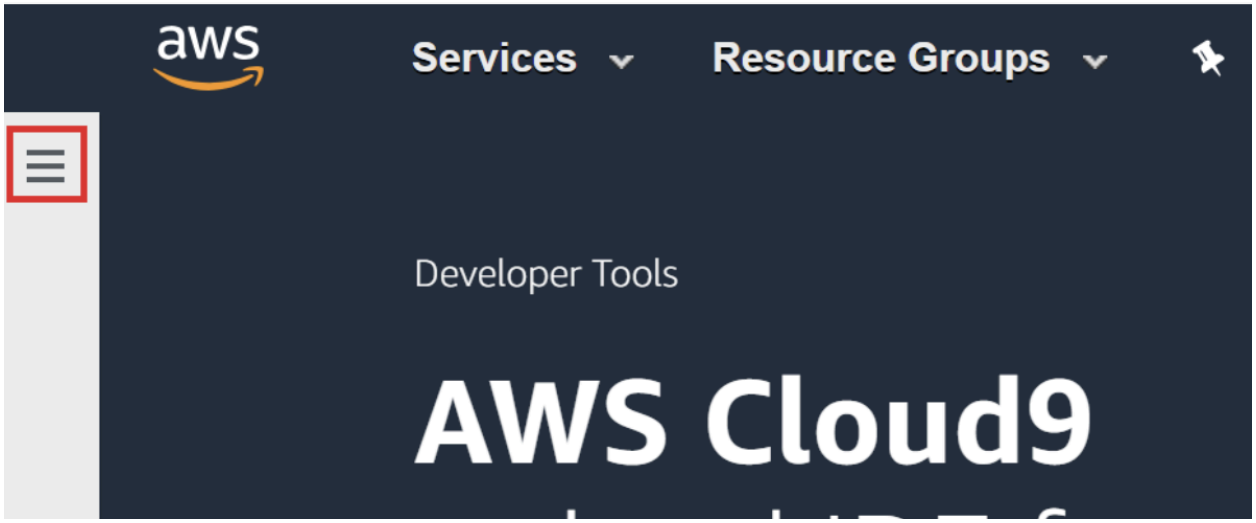
to save you the cost of running the instance when you are not using Cloud9. There is no additional cost to using Cloud9 beyond the cost of running the instance.

Instructions

1. In the AWS Console, enter *cloud9* in the search bar at the top,and click the **Cloud9** search result:



2. If you are not presented with the **Environments** view, click the sidebar accordion to show the sidebar menu, and select **Environments**:



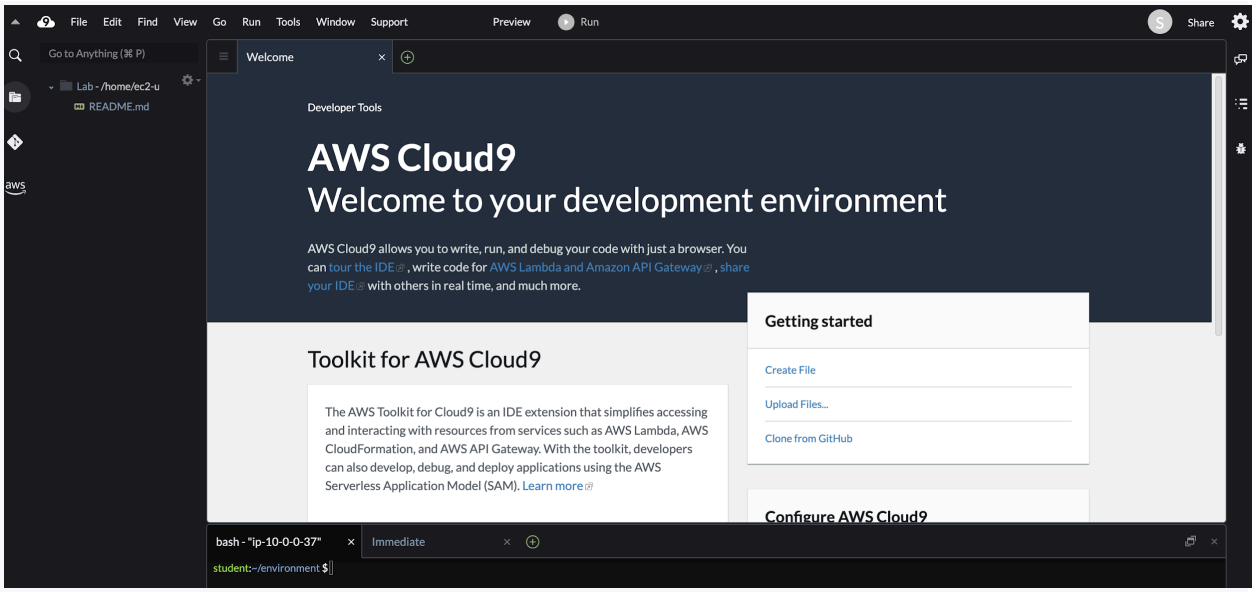
Warning: It can take a few minutes from the time you start the Lab until the Cloud9 environment is fully initialized and visible in **Your environments**. If you do not see the **Your environment** view, retry accessing the **Your environments** view every minute until it appears.

3. From the **Environments** menu, click **Open** under **Cloud9 IDE** to view the Cloud9 IDE:

Cloud9 IDE

Open

After a few seconds, you are presented with the Cloud9 IDE:



4. Take a minute to explore the Cloud9 interface. The default view shows three main panels:

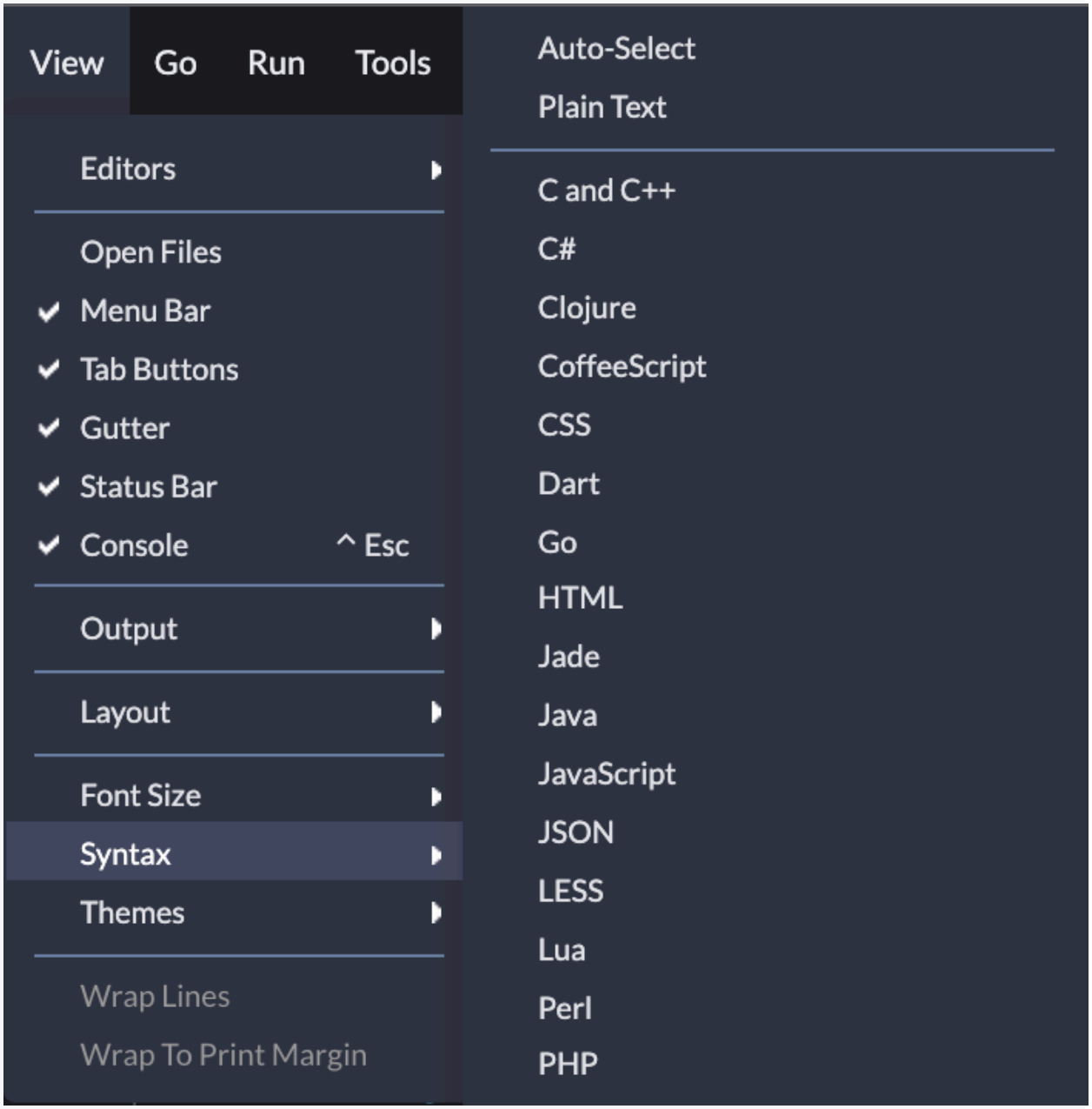
- **Environment**: Left panel that shows the file tree for your project
- **Editor**: Upper panel where you can open tabs for editing files. It displays a **Welcome** page by default.
- **Terminal**: Lower panel with a terminal connected to the EC2 instance backing the Cloud9 environment.

The default layout is only a suggestion. You can fully configure the layout. For example, you can add, remove, resize, and split panels; open files in the lower pane; and open terminals in the upper panel.

There is also a hidden pane on the right that can be opened by clicking on any of the **Collaborate**, **Outline**, **AWS Resources**, or **Debugger** tabs. The **Collaborate** tab shows the members of the Cloud9 environment, whether they are online or not, and their permissions, either read (**R**) or read/write (**RW**). You can enter messages in the chat log, and also navigate to the active cursor of other members by right-clicking their name and choosing show location.

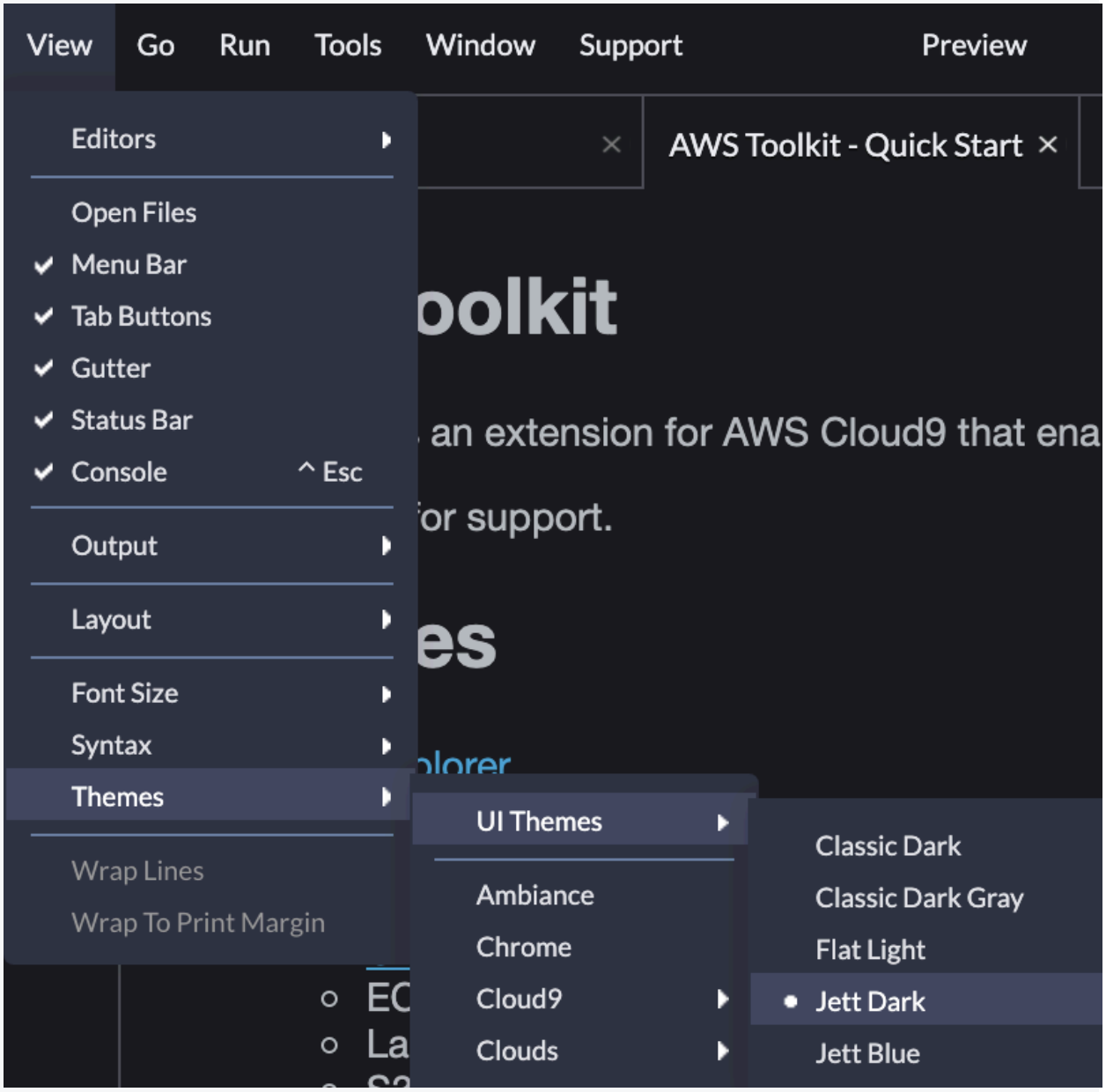
The **Outline** tab shows locations you can jump to in the current file, for example, the lines where functions are defined in code. **AWS Resources** is useful for managing Lambda functions. The **Debugger** is exactly what you would expect in an IDE - a tool for stepping through the code and inspecting values at runtime.

5. In the upper menu bar, select **View** > **Syntax** to see the list of languages with syntax highlighting support in Cloud9:



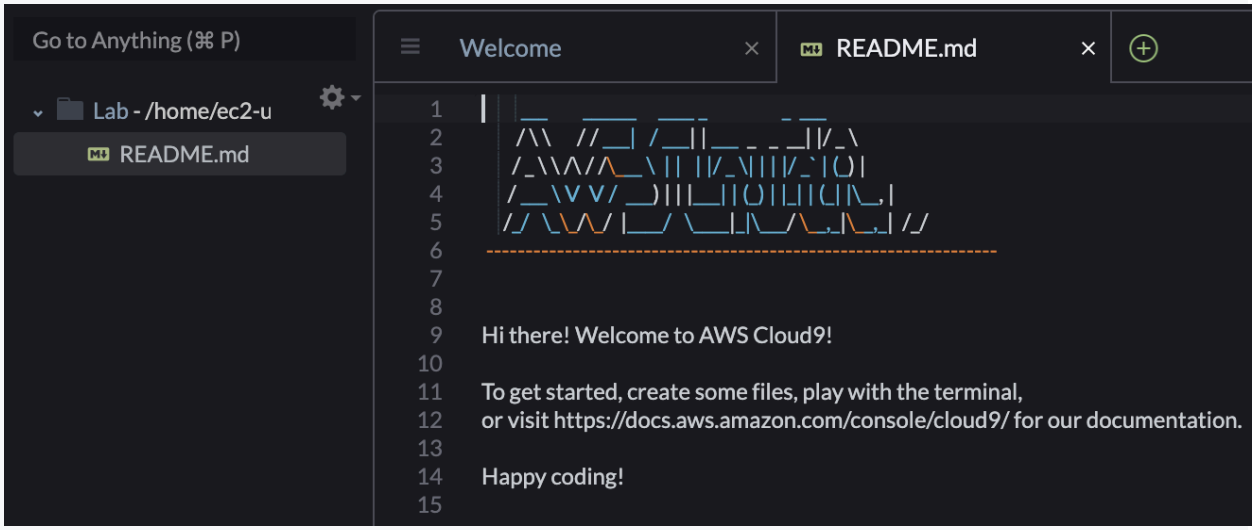
There are over 100 languages supported.

6. Navigate to **View** > **Themes** > **UI Themes** and try out a theme or two:



The following Lab images use the **Jett Dark** theme.

7. Double-click the **README.md** file in the **Environment** panel to open an editor tab for the file:



Cloud9 automatically creates a README.md file for new environments.

8. Select the **Environment** tab in the left panel, and close the **Welcome**,and **README.md** tabs in the upper panel.

Summary

In this Lab Step, you opened the AWS Cloud9 IDE and took a brief tour of its interface.

Installing the Required Tools

Introduction

AWS X-Ray is a service from Amazon that enables developers to analyze and debug production applications. It is particularly well suited to distributed systems and microservices.

In this lab step, you will install the tools required to trace a Java application:

- Java 11
- Maven
- X-Ray Daemon Tool

Instructions

1. Make sure you are in the Cloud9 IDE and in the default directory.
   At the bottom of the window, you will find the terminal window.
   If you are not sure which directory you are in, use this command in the terminal to change to the default directory:
2. Copy code
3. `cd /home/ec2-user/environment`
   The terminal window will allow you to enter Bash commands.
4. To install Java 11, enter the commands below in the terminal:
5. Copy code
6. `wget https://clouda-labs-assets.s3-us-west-2.amazonaws.com/aws-xray/openjdk-11%2B28_linux-x64_bin.tar.gz -O openjdk-11+28_linux-x64_bin.tar.gz`
7. `tar -xvf openjdk-11+28_linux-x64_bin.tar.gz`
8. `sudo mkdir -p /usr/lib/jvm`
9. `sudo mv jdk-11* /usr/lib/jvm/java-11-openjdk`
10. These commands download the Java 11 zip archive, unzip the archive and move the extracted java development kit to the `jvm` directory.
11. To configure your shell to use Java 11, enter the commands below:
12. Copy code
13. `export JAVA_HOME=/usr/lib/jvm/java-11-openjdk`
14. `export PATH=$JAVA_HOME/bin:$PATH`
15. `echo 'export JAVA_HOME=/usr/lib/jvm/java-11-openjdk' >> ~/.bash_profile`
16. `echo 'export PATH=$JAVA_HOME/bin:$PATH:$HOME/.local/bin:$HOME/bin' >> ~/.bash_profile`
17. `source ~/.bash_profile`
18. These commands add the path of Java 11 to your Bash terminal's `PATH` variable and reload the configuration of the Bash terminal.
    Adding these lines in the given file will change the default version of Java to Java 11.
19. To check the current version of Java, enter `java -version` and you will see the below output.
20.
```
openjdk version "11.0.28" 2025-07-15 LTS
OpenJDK Runtime Environment Corretto-11.0.28.6.1 (build 11.0.28+6-LTS)
OpenJDK 64-Bit Server VM Corretto-11.0.28.6.1 (build 11.0.28+6-LTS, mixed mode)
```
    Note: If you don't see version 11, go through the previous instructions again. Version 11 of Java is required for later lab steps.

```
bash - "ip-10-0-0-62.us-w ×   Immediate
jdk-11/lib/libverify.so
jdk-11/lib/libzip.so
jdk-11/lib/modules
jdk-11/lib/psfont.properties.ja
jdk-11/lib/psfontj2d.properties
jdk-11/lib/security/blacklisted.certs
jdk-11/lib/security/cacerts
jdk-11/lib/security/default.policy
jdk-11/lib/security/public_suffix_list.dat
jdk-11/lib/server/Xusage.txt
jdk-11/lib/server/libjsig.so
jdk-11/lib/server/libjvm.so
jdk-11/lib/src.zip
jdk-11/lib/tzdb.dat
jdk-11/release
student:~/environment $ sudo mkdir -p /usr/lib/jvm
student:~/environment $ sudo mv jdk-11* /usr/lib/jvm/java-11-openjdk
student:~/environment $ export JAVA_HOME=/usr/lib/jvm/java-11-openjdk
student:~/environment $ export PATH=$JAVA_HOME/bin:$PATH
student:~/environment $ echo 'export JAVA_HOME=/usr/lib/jvm/java-11-openjdk' >> ~/.bash_profile
student:~/environment $ echo 'export PATH=$JAVA_HOME/bin:$PATH:$HOME/.local/bin:$HOME/bin' >> ~/.bash_profile
student:~/environment $ source ~/.bash_profile
student:~/environment $ java -version
openjdk version "11.0.29" 2025-10-21 LTS
OpenJDK Runtime Environment Corretto-11.0.29.7.1 (build 11.0.29+7-LTS)
OpenJDK 64-Bit Server VM Corretto-11.0.29.7.1 (build 11.0.29+7-LTS, mixed mode)
student:~/environment $
```

21. Enter the below commands to install maven:
22. Copy code
23. `Wget https://archive.apache.org/dist/maven/maven-3/3.9.6/binaries/apache-maven-3.9.6-bin.tar.gz`
24. `tar -xzf apache-maven-3.9.6-bin.tar.gz`
25. `sudo mv apache-maven-3.9.6 /opt/maven`
26. `echo 'export MAVEN_HOME=/opt/maven' >> ~/.bash_profile`
27. `echo 'export PATH=$MAVEN_HOME/bin:$PATH' >> ~/.bash_profile`
28. `source ~/.bash_profile`

```
student:~/environment $ java -version
openjdk version "11.0.29" 2025-10-21 LTS
OpenJDK Runtime Environment Corretto-11.0.29.7.1 (build 11.0.29+7-LTS)
OpenJDK 64-Bit Server VM Corretto-11.0.29.7.1 (build 11.0.29+7-LTS, mixed mode)
student:~/environment $ wget https://archive.apache.org/dist/maven/maven-3/3.9.6/binaries/apache-maven-3.9.6-bin.tar.gz
--2025-11-13 14:01:54--  https://archive.apache.org/dist/maven/maven-3/3.9.6/binaries/apache-maven-3.9.6-bin.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a:a084::2
Connecting to archive.apache.org (archive.apache.org)|65.108.204.189|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9410508 (9.0M) [application/x-gzip]
Saving to: ‘apache-maven-3.9.6-bin.tar.gz’

100%[===================================================================================================>] 9,410,508    143KB/s   in 65s

2025-11-13 14:02:59 (142 KB/s) - ‘apache-maven-3.9.6-bin.tar.gz’ saved [9410508/9410508]

student:~/environment $ tar -xzf apache-maven-3.9.6-bin.tar.gz
student:~/environment $ sudo mv apache-maven-3.9.6 /opt/maven
student:~/environment $ echo 'export MAVEN_HOME=/opt/maven' >> ~/.bash_profile
student:~/environment $ echo 'export PATH=$MAVEN_HOME/bin:$PATH' >> ~/.bash_profile
student:~/environment $ source ~/.bash_profile
student:~/environment $ mvn -v
Apache Maven 3.9.6 (bc0240f3c744dd6b6ec2920b3cd08dcc295161ae)
Maven home: /opt/maven
Java version: 11.0.29, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-11-amazon-corretto.x86_64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "5.10.245-241.976.amzn2.x86_64", arch: "amd64", family: "unix"
student:~/environment $
```

29. Once the above commands are executed, you can check the maven version using the command `mvn -v`. You will see the below output:

```
Apache Maven 3.9.6 (bc0240f3c744dd6b6ec2920b3cd08dcc295161ae)
Maven home: /opt/maven
Java version: 11.0.28, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-11-amazon-corretto.x86_64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "5.10.239-236.958.amzn2.x86_64", arch: "amd64", family: "unix"
```

30. Confirm that the Java version is 11 and the Maven version is 3.9.6.
31. To install the X-Ray Daemon enter the commands below:
32. Copy code
33. `wget https://clouda-labs-assets.s3-us-west-2.amazonaws.com/aws-xray/aws-xray-daemon-linux-3.2.0.zip`
34. `unzip aws-xray-daemon-linux-3.2.0.zip`
35. Please note that you have only installed X-Ray Daemon and not started the Daemon which

```
student:~/environment $ wget https://clouda-labs-assets.s3-us-west-2.amazonaws.com/aws-xray/aws-xray-daemon-linux-3.2.0.zip
--2025-11-13 14:04:25--  https://clouda-labs-assets.s3-us-west-2.amazonaws.com/aws-xray/aws-xray-daemon-linux-3.2.0.zip
Resolving clouda-labs-assets.s3-us-west-2.amazonaws.com (clouda-labs-assets.s3-us-west-2.amazonaws.com)... 52.92.162.234, 52.92.165.34, 3.5.76.61, ...
Connecting to clouda-labs-assets.s3-us-west-2.amazonaws.com (clouda-labs-assets.s3-us-west-2.amazonaws.com)|52.92.162.234|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3795631 (3.6M) [application/zip]
Saving to: 'aws-xray-daemon-linux-3.2.0.zip'

100%[===================================================================================================================>] 3,795,631   --.-K/s   in 0.04s

2025-11-13 14:04:25 (88.1 MB/s) - 'aws-xray-daemon-linux-3.2.0.zip' saved [3795631/3795631]

student:~/environment $ unzip aws-xray-daemon-linux-3.2.0.zip
Archive:  aws-xray-daemon-linux-3.2.0.zip
  inflating: xray
  inflating: cfg.yaml
student:~/environment $
```

is required to send segments from your application to X-Ray.

Summary

In this lab step, you have installed the required tools for your application.

Configuring the AWS DynamoDB SDK and AWS X-Ray SDK

Introduction

Dependencies and libraries are required for any Application to work. In a normal Java application, you have to add libraries manually but Maven-based application helps you automate the process, you just have to add the dependency in **pom.xml** file and it will automatically download the **jars**.

In this lab step, you will be creating a Spring boot Application that will insert and fetch the data from Database, you will add the dependencies required for AWS DynamoDB and X-Ray.

Instructions

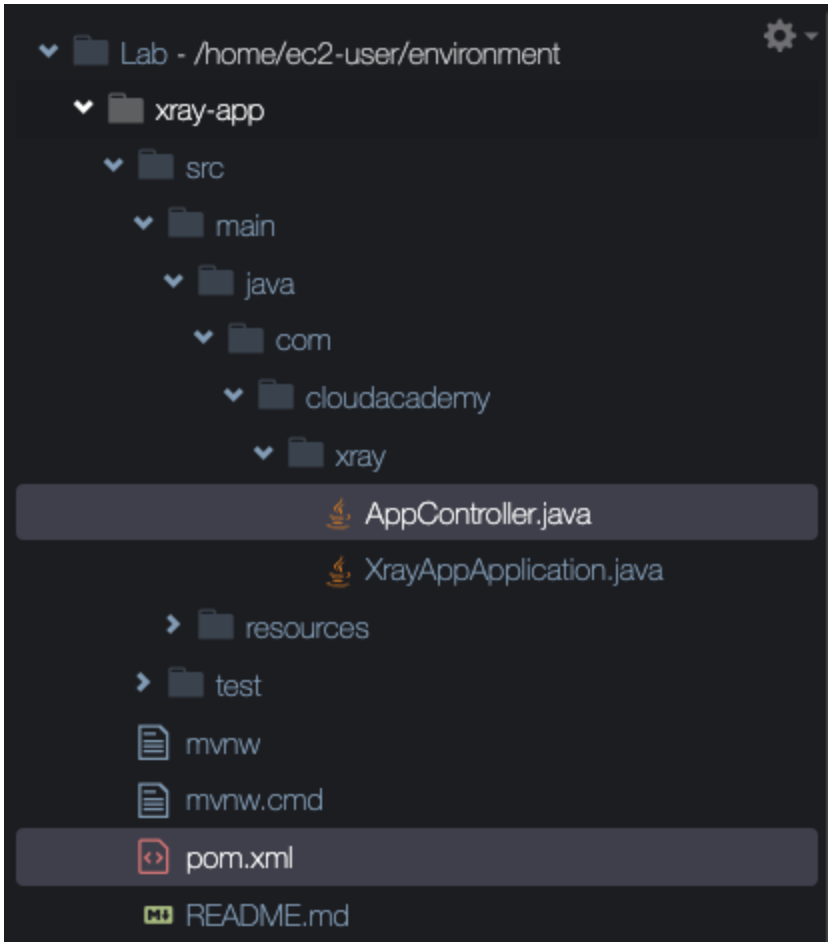
1. In terminal window enter:
2. Copy code
3. `git clone https://github.com/cloudacademy/xray-app`

   This command will clone the repository which has an application that already has methods to insert and fetch data to and from a database. The main agenda of this lab is to s

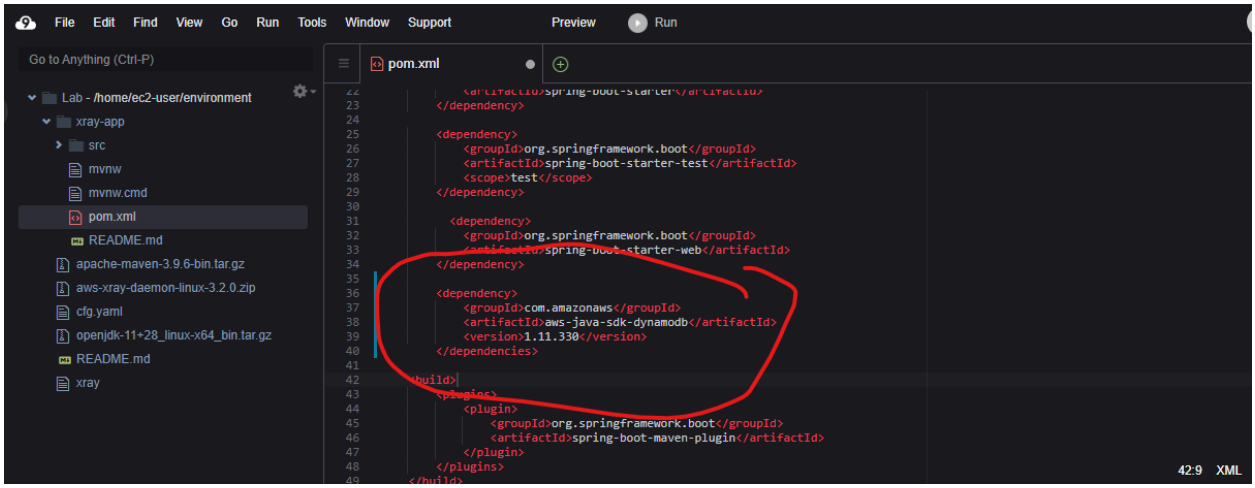
   ```
   student:~/environment $ unzip aws-xray-daemon-linux-3.2.0.zip
   Archive:  aws-xray-daemon-linux-3.2.0.zip
     inflating: xray
     inflating: cfg.yaml
   student:~/environment $ git clone https://github.com/cloudacademy/xray-app
   Cloning into 'xray-app'...
   remote: Enumerating objects: 27, done.
   remote: Counting objects: 100% (27/27), done.
   remote: Compressing objects: 100% (17/17), done.
   remote: Total 27 (delta 0), reused 27 (delta 0), pack-reused 0 (from 0)
   Receiving objects: 100% (27/27), 53.22 KiB | 4.43 MiB/s, done.
   student:~/environment $
   ```

   how how to configure AWS X-Ray in your application.

4. In the left sidebar, expand the **xray-app** directory by clicking on the **right arrow**. You will find **pom.xml** file in **xray-app** directory. Open the **pom.xml** file. You can refer to the directory as shown below:

5. 

6. Inside the **dependencies** element, add the following dependency XML snippet:
7. Copy code
8. ```
<dependency>
```
9. ```
    <groupId>com.amazonaws</groupId>
```
10. ```
    <artifactId>aws-java-sdk-dynamodb</artifactId>
```
11. ```
    <version>1.11.330</version>
```
12. ```
</dependency>
```
13. You can find **Add AWS SDK DynamoDB dependency here** in the **pom.xml** file, after this, paste
```



the dependency.

Note: You can press *Ctrl-Shift+B (Windows) or Cmd+Shift+B (Mac)* to format the file after adding the dependency.

The **pom.xml** file initially contains dependencies of Spring Boot tools. For this lab, you can add a dependency of the whole AWS SDK, but you should always add the dependency of a particular service that you need which in our case we need for DynamoDB. This will help you keep the size of your application low and hence help you get more latency.

14. Similarly, add dependencies for AWS X-Ray to the **dependencies** element:
15. Copy code
16. `<dependency>`
17. `  <groupId>com.amazonaws</groupId>`
18. `  <artifactId>aws-xray-recorder-sdk-core</artifactId>`
19. `  <version>2.8.0</version>`
20. `</dependency>`
21. `<dependency>`
22. `  <groupId>com.amazonaws</groupId>`

```
    <artifactId>aws-xray-recorder-sdk-aws-sdk</artifactId>
    <version>2.8.0</version>
</dependency>
<dependency>
    <groupId>com.amazonaws</groupId>
    <artifactId>aws-xray-recorder-sdk-aws-sdk-instrumentor</artifactId>
    <version>2.8.0</version>
</dependency>
```

31. For your reference, you can find **Add AWS X-Ray dependencies here** in the file, after this, you can paste the dependencies.

    aws-xray-recorder-sdk-core dependency is required for creating and transmitting segments.

    aws-xray-recorder-sdk-aws-sdk-instrumentor dependency is required to instrument any calls made using AWS SDK to AWS Services like you will make to DynamoDB. It will instrument all calls made from Java clients that you will initiate in further steps.

32. Press

```
        </dependencies>

        <dependency>
            <groupId>com.amazonaws</groupId>
            <artifactId>aws-xray-recorder-sdk-core</artifactId>
            <version>2.8.0</version>
        </dependency>

        <dependency>
            <groupId>com.amazonaws</groupId>
            <artifactId>aws-xray-recorder-sdk-aws-sdk</artifactId>
            <version>2.8.0</version>
        </dependency>

        <dependency>
            <groupId>com.amazonaws</groupId>
            <artifactId>aws-xray-recorder-sdk-aws-sdk-instrumentor</artifactId>
            <version>2.8.0</version>
        </dependency>

    <build>
        <plugins>
```

ctrl+s (Windows) or *cmd+s* (Mac) to **Save**.

Summary

In this lab step, you cloned a Spring Application having RESTful APIs to insert and fetch data from DynamoDB table. You have added the dependencies required for AWS DynamoDB, X-Ray.

Initiating the DynamoDB Client and Configuring the Segments

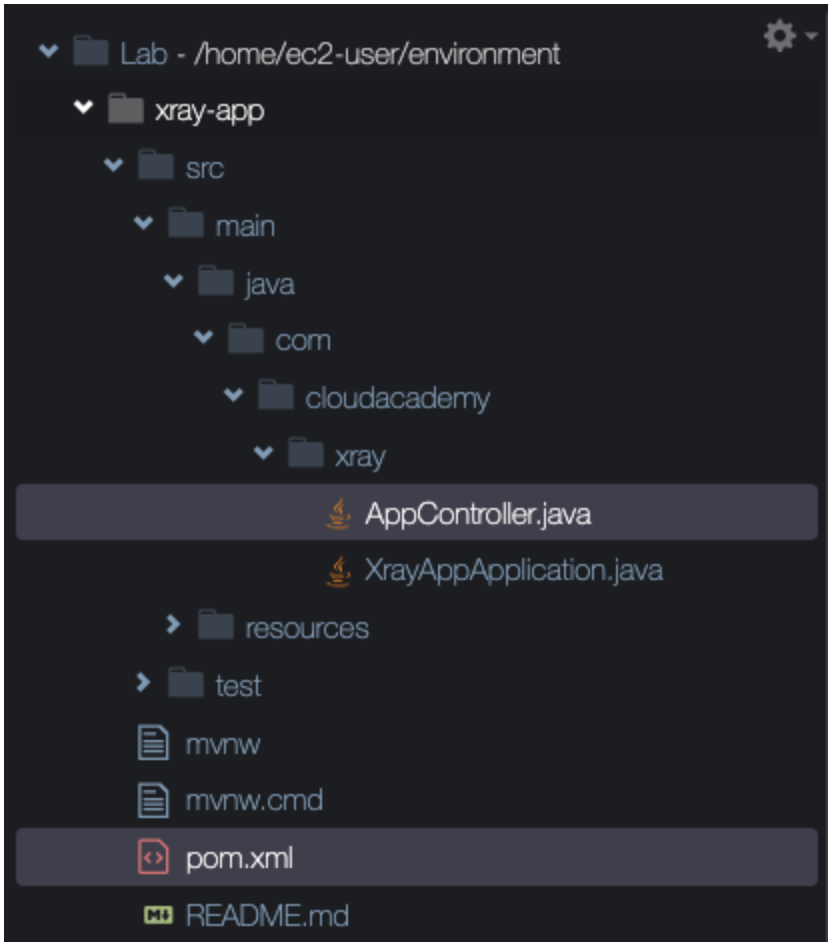
Introduction

AWS DynamoDB is a NoSQL Database. AWS SDK comes with predefined classes for many supported programming languages.

In this lab step, you will be initiating a DynamoDB Client and then configuring segments in your application to send traces of your application to X-Ray.

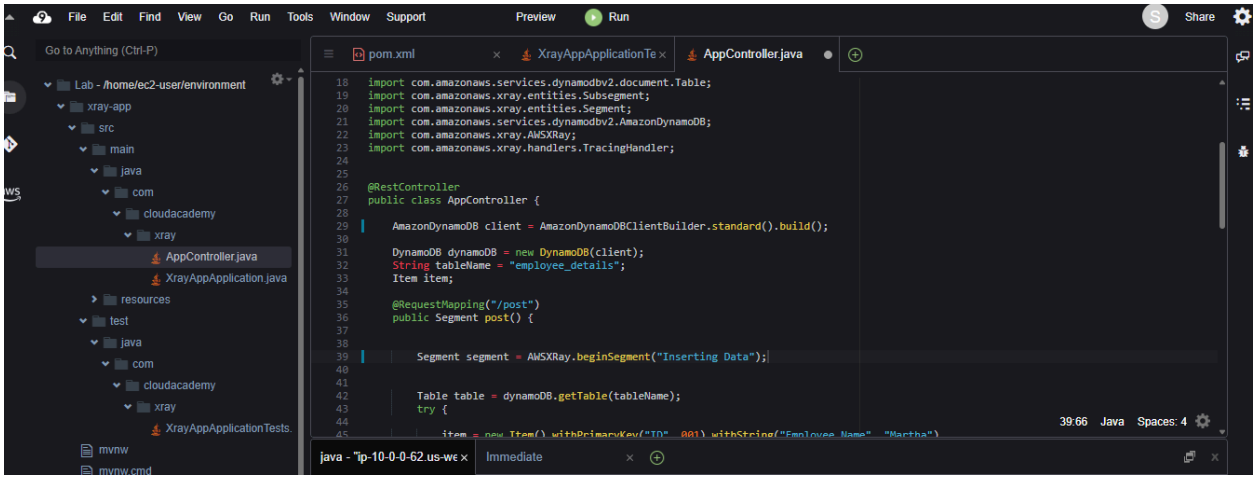
Instructions

1. Now, navigate to the **AppController.java** file.
   You have to expand the directory in following pattern **xray-app** > **src** > **main** > **java** > **com** > **cloudacademy** > **xray**.
   Refer to the directory as shown

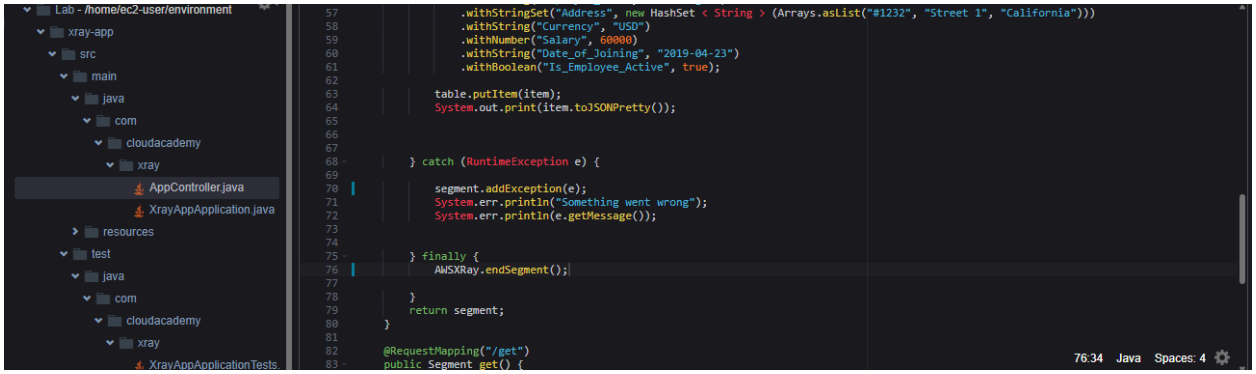
2. 

3. This file contains two APIs, one is RequestMapping(/post) to insert the data into the table, another one is RequestMapping(/get) to fetch the data from the table. Don't get confused with GetMapping to insert the data into the table, this is just to trigger the method inside the GetMapping API which is post() which itself uses the DynamoDB putItem() method that will take care of inserting the data. PostMapping is required if you don't use predefined methods of AWS SDK.
4. Where you find, **Initiate the DynamoDB Client**, add the following line of code.
5. Copy code
6. `AmazonDynamoDB client = AmazonDynamoDBClientBuilder.standard().build();`
7. The above line of code will initiate the DynamoDB client in your default region. A Client is always required to use a particular service in AWS SDK.

8. In the same file, where you find **Add the Begin Segment here for post method**, in the next line add the line of code:
9. Copy code
10. `Segment segment = AWSXRay.beginSegment("Inserting Data");`
11. It will start tracing your Post method but you must end the segment otherwise it might not work as expected.

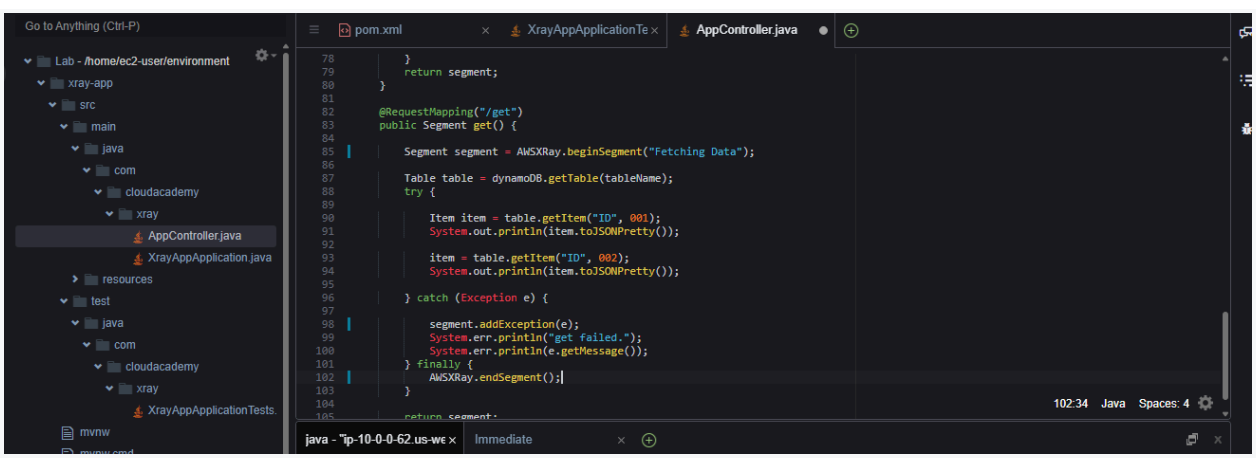


12. Where you find **Add the Segment add exception here for post method**, in the next line add the following line of code
13. Copy code
14. `segment.addException(e);`
15. It will trace all the exceptions occurring during execution.
16. Where you find **Add the End Segment here for post method**, in the next line add:
17. Copy code
18. `AWSXRay.endSegment();`
19. It will stop tracing your **Post** method. You should add separate segments for each method to distinguish among the requests.



20. Similarly, Repeat steps 3, 4, and 5 to add a segment for the **Get** method:

- Where you find **Add the Begin Segment here for Get method**. Add the following line of code:
- Copy code
- `Segment segment = AWSXRay.beginSegment("Fetching Data");`
- Where you find **Add the Segment add exception here for get method**, add the following line of code:
- Copy code
- `segment.addException(e);`
- Where you find **Add the End Segment here for get method**, add the following line of code:
- Copy code
- `AWSX`



`Ray.endSegment();`

Press *ctrl+s* (Windows) or *cmd+s* (Mac) to **Save**.

Summary

In this lab step, you have initiated the AWS DynamoDB client and configured segments in your application, you should now be able to trace your requests.

Starting X-Ray Daemon and Testing the Application

Introduction

AWS X-Ray is a cloud service that helps you trace requests to and from your applications. In this lab step, you will generate a jar for your application and then trigger your application using the APIs.

Instructions

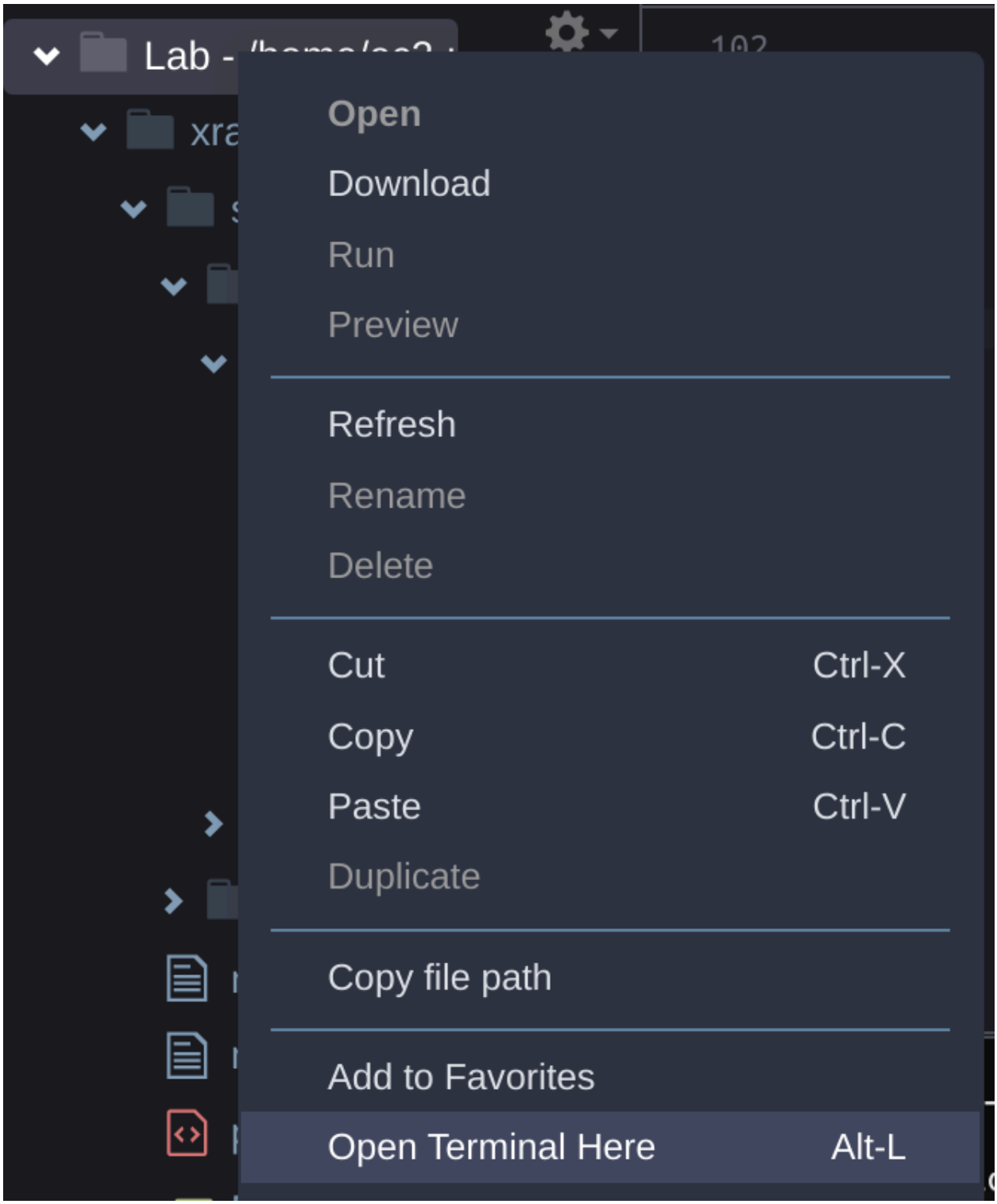
1. Navigate back to the terminal window. Make sure you are in the **default** directory. If you are not sure which directory you are in, use the following command in the terminal window to change to the default directory:
2. Copy code
3. `cd /home/ec2-user/environment`
4. Enter the following command to change the directory to your application:
5. Copy code
6. `cd xray-app`
7. Enter the command to download all the mentioned dependencies in the **pom.xml** file:
8. Copy code
9. `./mvnw package`
   This command will download all the dependencies and make them into a

package i.e. **jar** file.

Note: This might take around **7 minutes** to download all the dependencies.

```
student:~/environment/xray-app (main) $ ./mvnw package
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-starter-parent/2.4.4/spring-boot-starter-pare
nt-2.4.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-starter-parent/2.4.4/spring-boot-starter-paren
t-2.4.4.pom (8.6 kB at 10 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-dependencies/2.4.4/spring-boot-dependencies-2
.4.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-dependencies/2.4.4/spring-boot-dependencies-2.
4.4.pom (108 kB at 652 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/com/datastax/oss/java-driver-bom/4.9.0/java-driver-bom-4.9.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/com/datastax/oss/java-driver-bom/4.9.0/java-driver-bom-4.9.0.pom (4.1 kB at 54 kB/s
)
Downloading from central: https://repo.maven.apache.org/maven2/io/dropwizard/metrics/metrics-bom/4.1.18/metrics-bom-4.1.18.pom
Downloaded from central: https://repo.maven.apache.org/maven2/io/dropwizard/metrics/metrics-bom/4.1.18/metrics-bom-4.1.18.pom (5.3 kB at 54 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/io/dropwizard/metrics/metrics-parent/4.1.18/metrics-parent-4.1.18.pom
Downloaded from central: https://repo.maven.apache.org/maven2/io/dropwizard/metrics/metrics-parent/4.1.18/metrics-parent-4.1.18.pom (17 kB at 174
 kB/s)
Downloading from central: https://repo.maven.apache.o
```

10. In the left sidebar, at the top where you find **Lab - /home/ec2-user/environment,** right-click on it and click on **Open Terminal Here**:



This will open up a new terminal window as your previous terminal window will be used to run the application.

11. Enter the command below in the new terminal window which you just opened:

12. Copy code
13. `./xray -o -n us-west-2`

    You will see the below output stating the daemon has started:

```
bash - "ip-10-0-0-62.us-w ×   Immediate (Javascript (br ×   bash - "ip-10-0-0-62.us-w ×   bash - "ip-10-0-0-62.us-w ×
student:~/environment $ ./xray -o -n us-west-2
2025-11-13T14:39:47Z [Info] Initializing AWS X-Ray daemon 3.2.0
2025-11-13T14:39:47Z [Info] Using buffer memory limit of 9 MB
2025-11-13T14:39:47Z [Info] 144 segment buffers allocated
2025-11-13T14:39:47Z [Info] Using region: us-west-2
2025-11-13T14:39:47Z [Info] HTTP Proxy server using X-Ray Endpoint : https://xray.us-west-2.amazonaws.com
2025-11-13T14:39:47Z [Info] Starting proxy http server on 127.0.0.1:2000
```

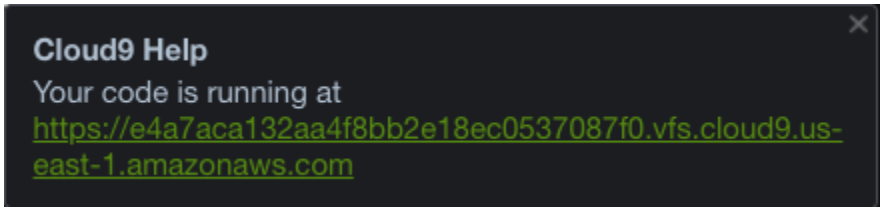
    This will send segments from your application to X-Ray.

    Leave X-Ray daemon running in this terminal.

14. In the previous terminal window, start your application using the below command:
15. Copy code
16. `java -jar target/xray-app-0.0.1-SNAPSHOT.jar`

    You should see the process running and at the end when you see the below output, then you are ready to go. This means your application has started and you can now trigger your APIs.
17. `Started Application in 7.008 seconds (JVM running for 8.383)`
18. At this stage, you will see a pop-up window on the top right corner of this terminal window.

    It will be similar to:



```
java - "ip-10-0-0-62.us-we ×   Immediate (Javascript (br ×   ./xray - "ip-10-0-0-62.us-w ×   bash - "ip-10-0-0-62.us-w ×

 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v2.4.4)

2025-11-13 14:41:06.381  INFO 15440 --- [           main] c.cloudacademy.xray.XrayAppApplicati
using Java 11.0.29 on ip-10-0-0-62.us-west-2.compute.internal with PID 15440 (/home/ec2-user/e
jar started by ec2-user in /home/ec2-user/environment/xray-app)
2025-11-13 14:41:06.391  INFO 15440 --- [           main] c.cloudacademy.xray.XrayAppApplication   : No active profile set, falling back to defau
lt profiles: default
2025-11-13 14:41:10.093  INFO 15440 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8080 (http)
2025-11-13 14:41:10.127  INFO 15440 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2025-11-13 14:41:10.128  INFO 15440 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.
44]
2025-11-13 14:41:10.268  INFO 15440 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationC
ontext
2025-11-13 14:41:10.268  INFO 15440 --- [           main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization c
ompleted in 3699 ms
2025-11-13 14:41:13.137  INFO 15440 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicationTas
kExecutor'
2025-11-13 14:41:13.634  INFO 15440 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http) with
context path ''
2025-11-13 14:41:13.660  INFO 15440 --- [           main] c.cloudacademy.xray.XrayAppApplication   : Started XrayAppApplication in 8.638 seconds
(JVM running for 9.981)

Cloud9 Help
Your code is running at
https://2e7bf0b9d56c4063a623b4b3b5be4e22.vfs.cloud9.us-west-2.amazonaws.com
```

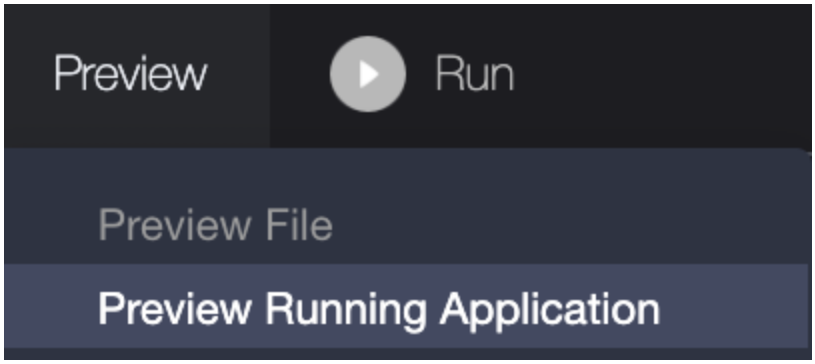
19. Click the link from the pop-up window to open in a new tab, append */post* at the end of the URL, and hit enter.

    You will see the output as your Segment Document:

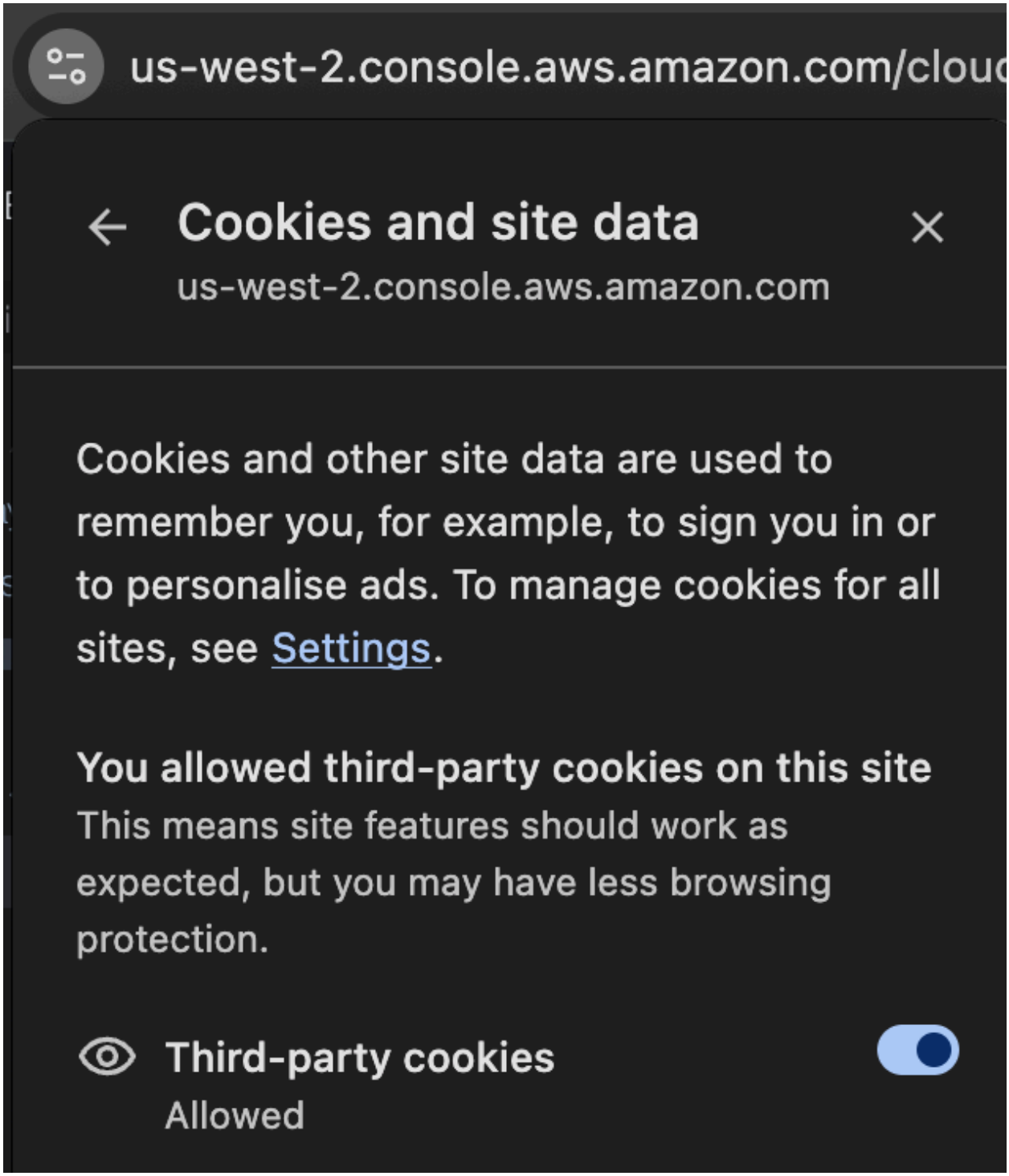

```
{"name":"Inserting Data","id":"4934b957b006161e","parentId":null,"startTime":1.618605569841E9,"traceId":"1-6079f601-
435fa134f1af787f48dc37d8","endTime":1.61860557064E9,"namespace":null,"subsegments":
[{"name":"AmazonDynamoDBv2","id":"7811158e703c906e","parentId":null,"startTime":1.618605569999E9,"endTime":1.618605570583E9,"namespace":"aws"
,"subsegments":[],"cause":{"workingDirectory":null,"id":null,"message":null,"paths":[],"exceptions":[]},"http":{"response":
{"content_length":2,"status":200}},"aws":
{"table_name":"employee_details","operation":"PutItem","request_id":"P5Q4KVQ7VT30PDGICL3DDNJQ93VV4KQNSO5AEMVJF66Q9ASUAAJG"},"sql":
{},"metadata":{},"annotations":{},"precursorIds":[]},
{"name":"AmazonDynamoDBv2","id":"6101adeb4dc60ccf","parentId":null,"startTime":1.618605570622E9,"endTime":1.618605570638E9,"namespace":"aws",
"subsegments":[],"cause":{"workingDirectory":null,"id":null,"message":null,"paths":[],"exceptions":[]},"http":{"response":
{"content_length":2,"status":200}},"aws":
{"table_name":"employee_details","operation":"PutItem","request_id":"N56OC4HM4G3PLTNVV7E0NRGIPFVV4KQNSO5AEMVJF66Q9ASUAAJG"},"sql":
{},"metadata":{},"annotations":{},"precursorIds":[]}],"cause":{"workingDirectory":null,"id":null,"message":null,"paths":[],"exceptions":
[]},"http":{},"aws":{"xray":{"sdk_version":"2.8.0","sdk":"X-Ray for Java"}},"sql":{},"metadata":{},"annotations":
{},"resourceArn":null,"user":null,"origin":null,"service":{"runtime":"OpenJDK 64-Bit Server VM","runtime_version":"11"}}
```


20.

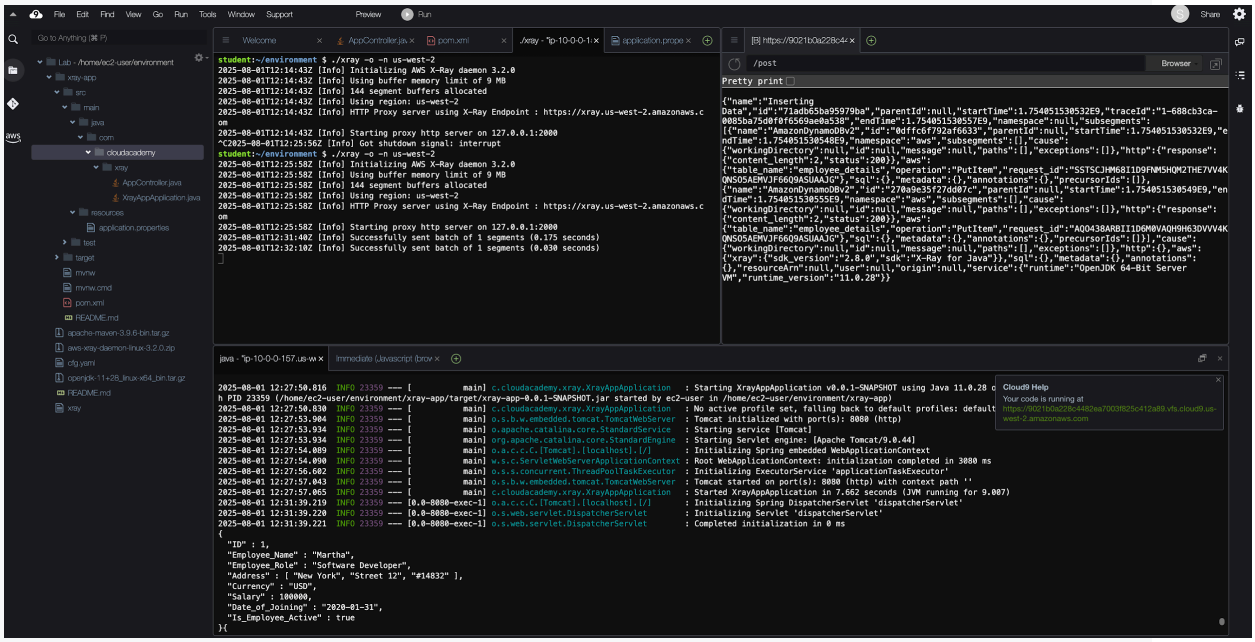
    Note: If you encounter an error when opening the link, it could be because Cloud9 is running in an incognito or private browsing window. To resolve this, go to **Preview** > **Preview Running Application** in the Cloud9 menu:



Next, make sure third-party cookies are enabled in your browser. For example, in Google Chrome, you can enable them by clicking the **View site information** icon (next to the address bar) > **Cookies and site data** > toggling on **Third-party cookies**:



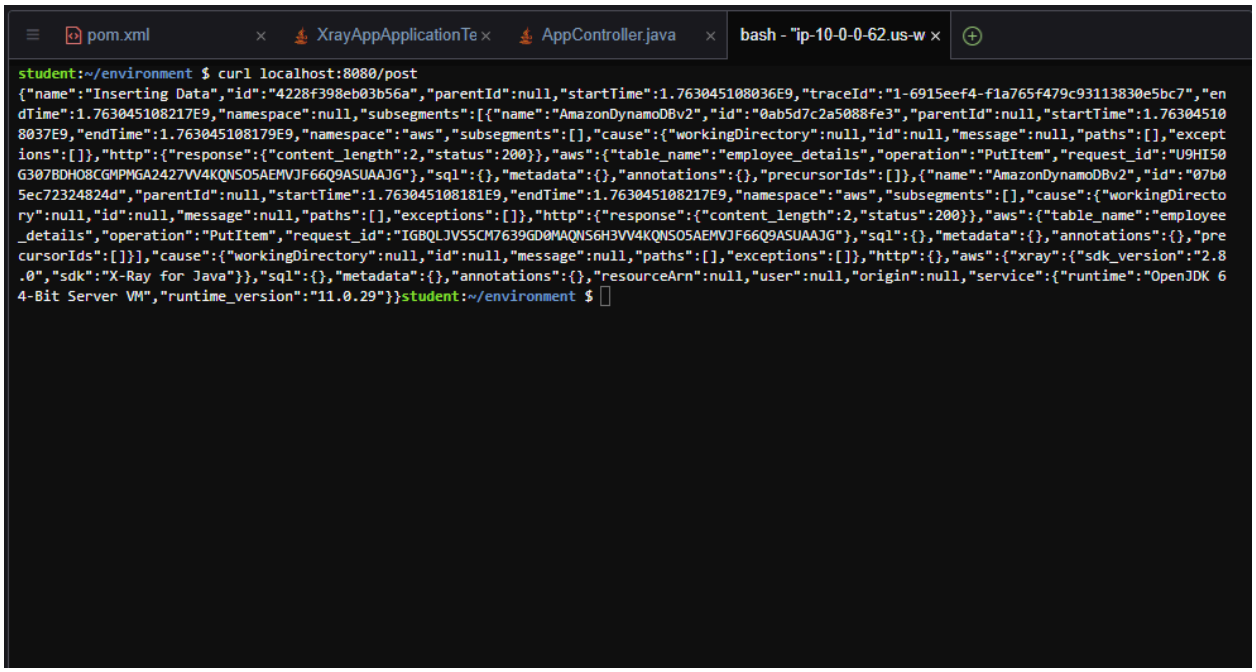
Your Cloud9 environment would look similar to:

21. 

22.

You have successfully inserted data into the DynamoDB table named `employee_details`. You can check the items you have inserted in the terminal window.

You can use this temporary link where your code is running, you cannot use localhost in your browser as this application is hosted in a virtual environment, but you can use `curl localhost:8080/post` to trigger the application from the terminal.



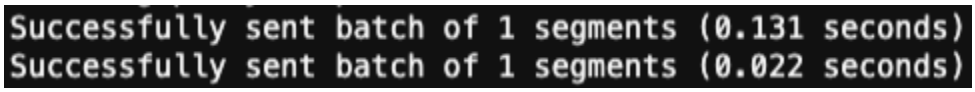
23. Again, append the same URL, use */get* instead of */post* and paste it in the new tab of your browser and hit *enter*.

    Again You will see the output as your Segment Document:

24.
```
{"name":"Fetching Data","id":"5d3a87525b2e0baa","parentId":null,"startTime":1.618605595379E9,"traceId":"1-6079f61b-
4a0fad6784d719862c958140","endTime":1.618605595416E9,"namespace":null,"subsegments":
[{"name":"AmazonDynamoDBv2","id":"3ea8032cf8edbb10","parentId":null,"startTime":1.61860559539E9,"endTime":1.618605595405E9,"namespace":"aws",
"subsegments":[],"cause":{"workingDirectory":null,"id":null,"message":null,"paths":[],"exceptions":[]},"http":{"response":
{"content_length":269,"status":200}},"aws":
{"table_name":"employee_details","operation":"GetItem","request_id":"VV6F00Q4TCS01K0TASQO1UH1JBVV4KQNSO5AEMVJF66Q9ASUAAJG"},"sql":
{},"metadata":{},"annotations":{},"precursorIds":[]},
{"name":"AmazonDynamoDBv2","id":"55fdcdf5de70ef4d","parentId":null,"startTime":1.618605595408E9,"endTime":1.618605595415E9,"namespace":"aws",
"subsegments":[],"cause":{"workingDirectory":null,"id":null,"message":null,"paths":[],"exceptions":[]},"http":{"response":
{"content_length":262,"status":200}},"aws":
{"table_name":"employee_details","operation":"GetItem","request_id":"KVMFHG3K41D99AK7S1BOMGIHP3VV4KQNSO5AEMVJF66Q9ASUAAJG"},"sql":
{},"metadata":{},"annotations":{},"precursorIds":[]}],"cause":{"workingDirectory":null,"id":null,"message":null,"paths":[],"exceptions":
[]},"http":{},"aws":{"xray":{"sdk_version":"2.8.0","sdk":"X-Ray for Java"}},"sql":{},"metadata":{},"annotations":
{},"resourceArn":null,"user":null,"origin":null,"service":{"runtime":"OpenJDK 64-Bit Server VM","runtime_version":"11"}}
```


    You have successfully fetched the data from the DynamoDB table named `employee_details`. You can check the output in the terminal window.

25. In the other terminal window where you have started the Daemon tool, you should see the output as below:

26. 

    You can see two outputs as one is for POST method means while inserting and another one for GET method means while fetching.

Summary

In this lab step, you have triggered the application to insert the data into the table, fetch the data from the table, and send trace segments to the AWS X-Ray service.

Finding and Understanding the Traces in the AWS Console

0/1

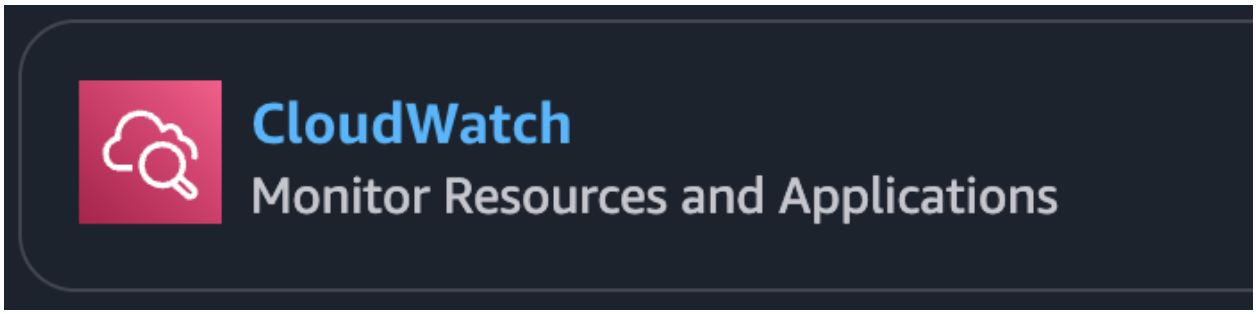
0 out of 1 validations checks passed

Introduction

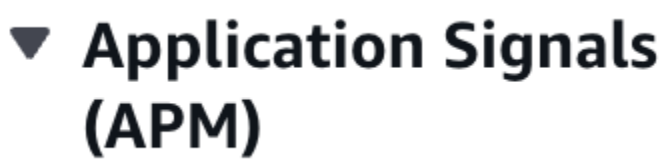
In this lab step, you will use the AWS X-Ray Console to view your traces.

Instructions

1. In the AWS Console, search for *CloudWatch* and click on **CloudWatch** under the services section.



2. On the left hand side navigation pane, select **Application Signals (APM)** > **Traces**:



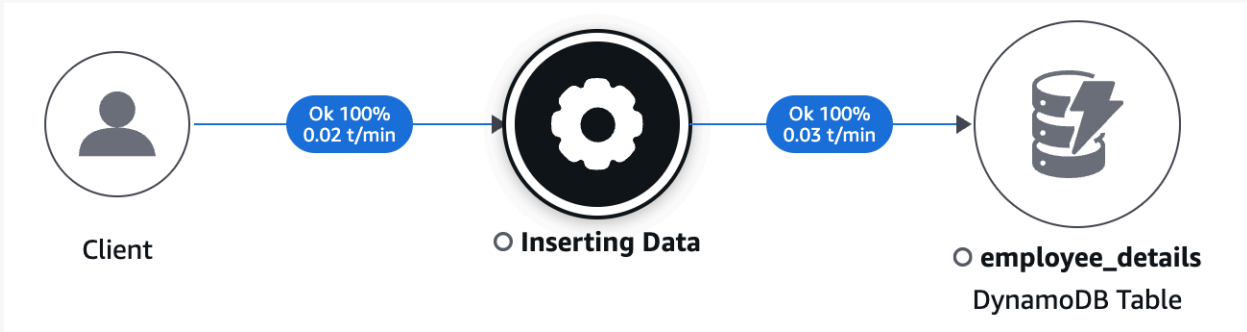
Traces

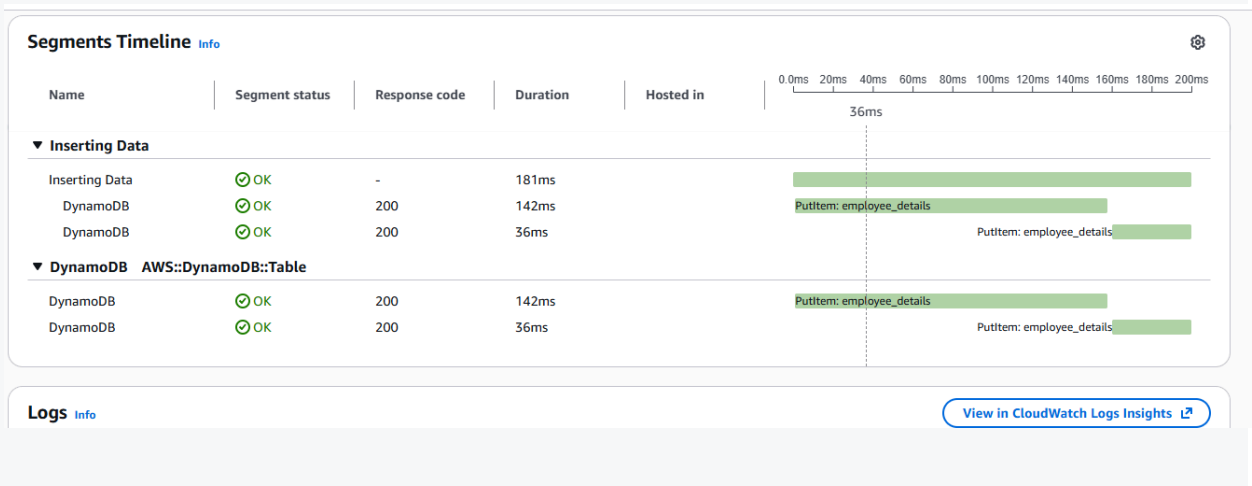
3. Change the time perioed to **1h** on the top right side:

4. Scroll down to the **Traces** section and you should see the two *post* and *get* traces:

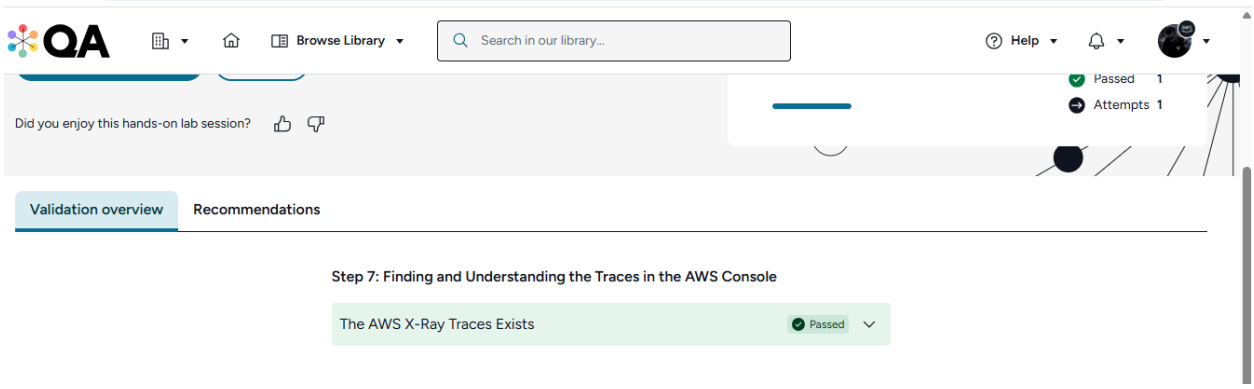
ID	Trace status	Response Time	Duration
...115341f8a2e2a4b81bcd75c5	OK	0.09s	0.09s
...d7a62633b4fbd086668e90c8	OK	0.036s	0.036s

5. Click on any of the traces to further understand the logging process





6. You can also select **Raw data** in the upper right corner to see all the methods called in a particular Segment like **putItem** or **getItem**.

Summary

In this lab step, you have found the traces in the AWS X-Ray console for the requests you made to the application